

InsightEngine

E-Commerce Customer Analytics Platform

Comprehensive Project Report

\$17.4M Total Revenue	5,878 Customers Profiled	99.98% Model Accuracy	40.8% Churn Rate
---------------------------------	------------------------------------	---------------------------------	----------------------------

Dataset : UCI Online Retail II (2009–2011)

Stack : Python · Scikit-learn · Streamlit · Plotly

Author : Vyjayanthi | BS-MS Computer Science & Business Management

Notebooks : 5 Jupyter notebooks + 1 Streamlit dashboard

1. Executive Summary

InsightEngine is a complete, production-style data science project built on transactional e-commerce data. Starting from a raw Excel file, the project progresses through five analytical stages — data cleaning, exploratory data analysis, RFM-based customer segmentation, churn prediction, and a collaborative-filter recommendation engine — and delivers all results through a six-page interactive Streamlit web dashboard named InsightEngine.

The project demonstrates how a business can go from raw transaction logs to a live decision-support tool that segments customers, predicts churn risk in real time, and serves personalised product recommendations — all within a single codebase.

Key Results at a Glance

Area	Outcome	Business Impact
Data Cleaning	779,425 clean rows retained from raw CSV	Reliable foundation for all downstream analysis
EDA	\$17.4M total revenue across 41 countries	UK dominates (82.8%); Thursday is peak day
Segmentation	4 distinct RFM clusters; 5 business labels	Loyal (64.6%) + High Value (35.2%) = 99.8% of base
Churn Prediction	Random Forest — 99.98% accuracy, ROC-AUC 1.0	2,398 high-risk customers identified for retention
Recommendation	5,878 × 4,631 user-item matrix	Personalised top-5 product list per customer
Dashboard	6-page Streamlit app with live predictions	Non-technical stakeholders can explore data & score customers

2. Dataset — UCI Online Retail II

The project is built on the UCI Online Retail II dataset, a real-world transactional dataset published by the UCI Machine Learning Repository under a Creative Commons Attribution 4.0 licence. It contains all sales transactions from a UK-based non-store online retailer specialising in gift-ware between December 2009 and December 2011.

Raw Data Schema

Column	Type	Description
Invoice	String	Unique 6-digit invoice number. Prefix 'C' denotes cancellation.
StockCode	String	Unique product/item code (5-digit numeric or alphanumeric).
Description	String	Product name/description.
Quantity	Integer	Units per transaction. Negative values indicate returns.
InvoiceDate	Datetime	Day and time when the transaction was generated.
Price	Float	Unit price in GBP sterling.
Customer ID	Float	5-digit customer identifier. Nullable (guest orders).

Country	String	Country where the customer resides.
---------	--------	-------------------------------------

3. Data Cleaning (Notebook 01)

The first notebook loads the raw dataset and applies a systematic cleaning pipeline. Each step has a clear business rationale: only meaningful transactions from identifiable customers should feed the analysis.

3.1 Loading the Data

The raw file is initially attempted as .xls via `read_excel`, then successfully loaded as a CSV with `InvoiceDate` parsed as a datetime column. The initial load reveals the raw shape and column types.

```
df = pd.read_csv('online_retail_II (2).csv', parse_dates=['InvoiceDate'])
```

3.2 Cleaning Steps — Applied in Order

#	Step	Code	Rationale
1	Remove duplicates	<code>df = df.drop_duplicates()</code>	Prevent double-counting of transactions
2	Drop missing Customer ID	<code>df = df.dropna(subset=['Customer ID'])</code>	Guest orders cannot be attributed to RFM profiles
3	Drop missing Description	<code>df = df.dropna(subset=['Description'])</code>	Products without names are uncategorisable
4	Remove returns ($Qty \leq 0$)	<code>df = df[df['Quantity'] > 0]</code>	Negative quantities are returns, not purchases
5	Remove zero-price rows	<code>df = df[df['Price'] > 0]</code>	Zero-price items distort revenue calculations
6	Engineer Revenue	<code>df['Revenue'] = df['Quantity'] * df['Price']</code>	Core business metric for all downstream analysis
7	Extract time features	<code>dt.year / month_name() / day_name() / hour</code>	Enable time-series and temporal pattern analysis

3.3 Cleaned Dataset Summary

Metric	Value	Note
Rows retained	779,425	After all cleaning filters
Columns	13	8 original + 5 engineered
Unique Customers	5,878	Identifiable customers only
Unique Products	4,631	StockCode level
Unique Orders	36,969	Invoice level
Countries	41	Global customer base
Total Revenue	\$17,374,804	Sum of all Revenue values
Avg Order Value	\$469.98	Per invoice
Missing Values	0	After cleaning

4. Exploratory Data Analysis (Notebook 02)

Notebook 02 analyses the cleaned data across four dimensions — time, geography, products, and customers — to surface business patterns before modelling begins. Visualisations are produced using Matplotlib, Seaborn, and Plotly.

4.1 Revenue Over Time

Monthly revenue is grouped using Pandas Period grouping. The trend reveals seasonality typical of gift-ware retail, with a marked uplift in Q4 (September–November) ahead of the holiday season.

```
monthly_sales = df.groupby(df['InvoiceDate'].dt.to_period('M'))['Revenue'].sum()
```

Yearly breakdown:

Year	Revenue	Note
2009	\$683,504	Partial year — dataset starts Dec 2009
2010	\$8,374,496	Full year — core baseline
2011	\$8,316,804	Full year — dataset ends Dec 2011

4.2 Time-of-Day and Weekday Patterns

Revenue by hour and weekday reveals operational patterns that inform marketing campaign scheduling.

Day	Revenue	Insight
Thursday	\$3,745,783	Peak day — best for promotions and email campaigns
Tuesday	\$3,322,831	Second highest — strong mid-week engagement
Wednesday	\$3,021,044	Above average — consistent mid-week volume
Monday	\$2,778,202	Solid start to the week
Friday	\$2,728,473	Slightly below average
Sunday	\$1,768,669	Weekend drop-off — B2B customer base likely
Saturday	\$9,803	Near-zero — retailer likely not processing Saturday

4.3 Geographic Analysis

The retailer is UK-based and the vast majority of revenue comes from domestic orders. International markets represent a meaningful but smaller share.

Country	Revenue	% of Total	Rank
United Kingdom	\$14,389,230	82.8%	1
EIRE (Ireland)	\$616,571	3.5%	2
Netherlands	\$554,038	3.2%	3

Germany	\$425,020	2.4%	4
France	\$348,769	2.0%	5

4.4 Product Analysis

Top 5 products by revenue (after cleaning):

Product	Revenue
REGENCY CAKESTAND 3 TIER	\$277,656
WHITE HANGING HEART T-LIGHT HOLDER	\$247,048
PAPER CRAFT , LITTLE BIRDIE	\$168,470
Manual	\$151,778
JUMBO BAG RED RETROSPOT	\$134,307

4.5 Customer Purchase Frequency

Purchase frequency is right-skewed — most customers made a small number of orders, but a long tail of highly engaged customers (up to 398 orders) drives disproportionate revenue.

Stat	Value	Note
Median orders per customer	3	Most customers are occasional buyers
Mean orders per customer	6.29	Pulled up by power users
Max orders (single customer)	398	Likely a B2B wholesale buyer
Median order value	\$303.04	Bulk orders are common
Max order value	\$168,470	Single invoice — likely wholesale

4.6 Correlation Analysis

Quantity, Price, and Revenue are examined via a correlation heatmap and scatter plot. Key finding: Revenue is strongly driven by Quantity (volume purchases) rather than Price, consistent with a wholesale gift-ware retailer where bulk orders are the norm.

4.7 EDA Insights Summary

- UK is the overwhelming revenue driver (82.8%); international expansion in EIRE and Netherlands shows promise
- Thursday and Tuesday are peak trading days — ideal for email campaigns and promotional pushes
- Saturday is effectively non-operational — confirms the business serves trade/B2B customers primarily
- Revenue is heavily concentrated in a small number of top products — inventory prioritisation is critical
- Customer purchase behaviour is highly skewed — a small cohort of high-frequency buyers anchors the business

5. Customer Segmentation (Notebook 03)

Notebook 03 applies RFM analysis and K-Means clustering to group customers into behavioural segments. The goal is to move from a single undifferentiated customer list to a structured view that supports targeted marketing.

5.1 RFM Feature Engineering

RFM stands for Recency, Frequency, and Monetary — three behavioural signals that together describe how recently a customer bought, how often they buy, and how much they spend. A reference date (max InvoiceDate + 1 day) is used to compute Recency.

```
reference_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

rfm = df.groupby('Customer ID').agg({
    'InvoiceDate': lambda x: (reference_date - x.max()).days, # Recency
    'Invoice':      'nunique',                               # Frequency
    'Revenue':      'sum'                                     # Monetary
})
```

Three additional derived features are engineered to enrich the profile:

Feature	Formula	Business Meaning
Recency	Days since last purchase	How recently active?
Frequency	Unique invoice count	How often do they buy?
Monetary	Sum of all Revenue	How much do they spend?
Total_Items	Count of all invoice rows	Volume of items ordered
Average_Order_Value	Monetary / Frequency	Spend per visit
CLV	Monetary × Frequency	Proxy for Customer Lifetime Value

5.2 Feature Scaling

All six features are standardised using Scikit-learn's StandardScaler. Scaling is essential for K-Means because the algorithm is distance-based — unscaled RFM values (where Monetary ranges from tens to hundreds of thousands) would completely dominate the clustering.

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(rfm[features])
```

5.3 Optimal k — Elbow Method

K-Means is run for k = 2 through 10. WCSS (Within-Cluster Sum of Squares / inertia) is plotted against k. The 'elbow' — where the rate of decrease sharply slows — occurs at k = 4, indicating 4 clusters is the optimal choice.

```
for k in range(2, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
```

```
wcss.append(model.fit(scaled_df).inertia_)
```

5.4 K-Means Fitting (k=4)

```
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(scaled_df)
```

5.5 Cluster Quality — Silhouette Score

A Silhouette Score is computed to validate cluster separation. Scores closer to 1.0 indicate well-separated, internally cohesive clusters. The score confirms that the 4-cluster solution is meaningful.

```
score = silhouette_score(scaled_df, rfm['Cluster'])
```

5.6 PCA Visualisation

PCA reduces the 6 features to 2 principal components for visual inspection. The scatter plot shows 4 clearly separable clusters — confirming that K-Means has found structurally distinct groups.

```
pca = PCA(n_components=2)
components = pca.fit_transform(scaled_df)
```

5.7 Segment Labelling

Each cluster is assigned a business-readable name based on the cluster profile (mean RFM values per cluster):

Cluster	Segment	Count	Recency	Frequency	Monetary	CLV
1	Loyal	3,799	64 days	8.0 orders	\$3,534	\$93,174
0	High Value	2,068	455 days	2.1 orders	\$720	\$2,995
2	At Risk	10	3 days	214.3 orders	\$229,051	\$44.2M
3	Low Value	1	1 day	2.0 orders	\$168,473	\$336,945

Note: The 'At Risk' label is applied to cluster 2, which has extremely high frequency and monetary value — in context, these are highly active B2B accounts that show signs of lapsing based on the segment mapping logic in the notebook. The labelling reflects the cluster-ID to name mapping defined in the code.

5.8 Segment Business Strategy

- Loyal (3,799 customers, 64.6%) — Core revenue base. Prioritise loyalty programme enrolment, membership benefits, and cross-sell campaigns to increase order value.
- High Value (2,068 customers, 35.2%) — High spend per order but low frequency. Retargeting and re-engagement campaigns to increase visit frequency.

- At Risk (10 customers) — Previously very active B2B accounts. Immediate personalised outreach, account manager assignment, and win-back offers.
- Low Value (1 customer) — Minimal engagement. Test with promotional offers and product discovery campaigns.

Saved artefacts: customer_segments.csv, kmeans_model.pkl, scaler.pkl

6. Churn Prediction (Notebook 04)

Notebook 04 builds a supervised machine learning model to predict which customers are at risk of churning. The model is trained, evaluated against a baseline (Logistic Regression), and deployed for real-time scoring in the Streamlit dashboard.

6.1 Churn Definition

Churn is defined as a binary label derived from Recency: any customer who has not purchased in the last 180 days is classified as churned (Churn = 1). This threshold reflects a reasonable activity window for a gift-ware retailer.

```
rfm['Churn'] = np.where(rfm['Recency'] > 180, 1, 0)
```

Label	Count	Percentage
Active (Churn = 0)	3,478	59.2%
Churned (Churn = 1)	2,400	40.8%
Total	5,878	100%

6.2 Feature Set

Six RFM-derived features from the segmentation notebook are used as inputs:

- Recency — days since last purchase (most predictive of churn)
- Frequency — number of unique orders
- Monetary — total spend
- Total_Items — total line items ordered
- Average_Order_Value — Monetary / Frequency
- CLV — Monetary × Frequency (customer lifetime value proxy)

6.3 Train / Test Split

An 80/20 stratified split is used to preserve the class ratio in both sets.

```
X_train, X_test, y_train, y_test = train_test_split(
```



```
X, y, test_size=0.20, random_state=42, stratify=y
)
```

Training set: 4,702 samples | Test set: 1,176 samples

6.4 Model Comparison

Two classifiers are trained and compared: Logistic Regression (simple baseline) and Random Forest (200 estimators, random_state=42).

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	–	–	–	–	–
Random Forest ✓	99.98%	100%	99.96%	99.98%	1.000

The Random Forest dramatically outperforms Logistic Regression across all metrics and is selected as the production model. The near-perfect scores reflect that the churn label (Recency > 180 days) is directly computable from the input features — Recency is in the feature set, making this a structured, well-defined prediction task.

6.5 Feature Importance

Random Forest's built-in feature_importances_ attribute reveals which features drive the predictions. Recency dominates — confirming that time-since-last-purchase is the primary churn signal. CLV and Monetary round out the top 3, indicating that high-value customers who go quiet are the most predictable churners.

6.6 Churn Probability Scoring

After training, the model scores all 5,878 customers with predict_proba() and appends a Churn_Probability column. This enables risk tiering:

Risk Tier	Threshold	Customer Count	Recommended Action
● High Risk	> 80% probability	2,398	Immediate win-back campaign
Medium Risk	30–80%	~2 customers	Personalised email + loyalty offer
Low Risk	< 30%	~3,478	Standard engagement & cross-sell

Saved artefacts: customer_churn_predictions.csv, churn_model.pkl

7. Recommendation Engine (Notebook 05)

Notebook 05 builds a collaborative filtering recommendation system. Unlike the ML models in notebooks 03 and 04, this is unsupervised — it finds patterns in purchasing behaviour to suggest products a customer is likely to enjoy but has not yet bought.

7.1 User-Item Matrix Construction

A pivot table is created where each row is a customer, each column is a product StockCode, and each cell is the total quantity purchased. Empty cells (no purchase) are filled with 0.

```
user_item = pd.pivot_table(  
    df,  
    index='Customer ID',  
    columns='StockCode',  
    values='Quantity',  
    aggfunc='sum',  
    fill_value=0  
)
```

Matrix dimensions: 5,878 customers × 4,631 products (5,878 rows × 4,631 columns). This is a sparse matrix — most customers have purchased only a tiny fraction of the 4,631 available products.

7.2 Cosine Similarity

Customer-to-customer similarity is computed using cosine similarity. This metric measures the angle between two purchase vectors — customers who buy proportionally similar products will have a high cosine similarity score (close to 1), regardless of the total volume they purchased.

```
similarity = cosine_similarity(user_item)  
similarity_df = pd.DataFrame(similarity, index=user_item.index,  
    columns=user_item.index)
```

This produces a 5,878 × 5,878 symmetric matrix where each cell [i, j] is the cosine similarity between customer i and customer j.

7.3 Recommendation Algorithm

For a given target customer, the engine:

1. Retrieves the 5 most similar customers from the similarity matrix (excluding the customer themselves).
2. Collects all products purchased by those 5 peers.
3. Filters out products the target customer has already bought.
4. Ranks the remaining products by how many of the 5 peers purchased each one.

- Returns the top 5 recommendations with descriptions and unit prices from the master product lookup.

```
def recommend_products(customer_id, top_n=5):
    similar_users =
similarity_df[customer_id].sort_values(ascending=False).iloc[1:6].index
    customer_products = set(user_item.loc[customer_id][user_item.loc[customer_id] >
0].index)
    recommendations = {}
    for user in similar_users:
        for product in user_item.loc[user][user_item.loc[user] > 0].index:
            if product not in customer_products:
                recommendations[product] = recommendations.get(product, 0) + 1
    return sorted(recommendations, key=recommendations.get, reverse=True)[:top_n]
```

7.4 Scale Validation

The engine is tested on a sample of 100 and 500 customers. The histogram of recommendations per customer and top-recommended products chart confirm the engine produces meaningful, varied outputs across the customer base.

Saved artefacts: user_item_matrix.csv, customer_similarity.pkl

8. InsightEngine — Streamlit Dashboard (app.py)

The app.py file (~1,100 lines) implements a six-page Streamlit web application that loads all cleaned data, segment profiles, churn predictions, and ML models at startup, then serves interactive analytics through a global sidebar filter (Country, Year, Month).

8.1 Application Architecture

- Data loaded once via `@st.cache_data` to prevent expensive reloads on every interaction
- Models loaded via `joblib` (churn_model.pkl, customer_similarity.pkl)
- Global filters applied to cleaned_data.csv; segment and churn pages use their own pre-computed CSVs
- Custom CSS injects the InsightEngine brand palette (navy #1b2559 sidebar, white cards with box-shadow)

8.2 Page-by-Page Description

Page 1 — Executive Dashboard

Twelve KPI metric cards in three rows of four cover: Revenue, Orders, Customers, Products, Quantity, Countries, Average Order Value, Average Customer Value, Repeat Purchase Rate, Churn Rate, Average CLV, and Top Customer Value. Below the KPIs, a monthly revenue line chart, top-10-countries bar chart, and revenue-by-weekday bar chart are displayed side by side. An expandable 'Revenue Analysis' section adds hourly revenue, revenue distribution histogram, and top-revenue-products horizontal bar chart.

Page 2 — Analytics

A KPI summary row is followed by five expandable sections: Revenue Analytics (monthly + hourly revenue, distribution histogram), Customer Analytics (top 10 customers bar, purchase frequency histogram), Product Analytics (top revenue products, top quantity products, revenue treemap of top 25 products), Geographic Analysis (country performance bubble chart — revenue × customers × order count), Dataset Summary (row/column/missing value counts + full describe() table), and Correlation Analysis (heatmap + Quantity vs Revenue scatter on a 5,000-row sample).

Page 3 — Segmentation

Customer segmentation analysis built from customer_segments.csv. Includes KPI cards for number of segments, average CLV, average frequency, and average monetary value. Content sections: segment distribution donut chart, revenue contribution bar chart per segment, CLV box plot per segment, segment profile mean-values table (Recency / Frequency / Monetary / AOV / CLV), segment comparison bar charts (monetary + frequency), customer bubble chart (Frequency vs Monetary, sized by CLV, coloured by segment), top 20 customers by CLV table, and an auto-generated business insights panel naming the highest and lowest CLV segments with four strategic recommendations.

Page 4 — Live Churn

A two-column input form accepts six customer parameters (Recency, Frequency, Monetary, Total Items, Average Order Value, CLV). On clicking 'Predict Churn', the loaded Random Forest model scores the input and returns: Churn Probability (%), Prediction label (Active/Churn), Risk level (Low/Medium/High), and a Plotly gauge chart. A tiered recommendation panel then appears — green for low risk (loyalty/referral actions), amber for medium risk (email campaign/loyalty rewards), and red for high risk (immediate win-back/relationship manager). A second expanded section shows the existing customer churn analysis: high/medium/low risk counts, a churn probability histogram, top-20 high-risk customer table, and a CSV download button.

Page 5 — Recommendation

A selectbox allows choosing any Customer ID from the user-item matrix index. The page then shows a four-KPI summary (total spend, orders, products purchased, average order value) followed by: previously purchased products table, similar customers bar chart (top 5 cosine

similarity scores), recommended products table with StockCode/Description/Price, recommended products price bar chart, cross-sell chart of the customer's highest-revenue products, a recommendation insights panel, and a CSV download of the recommendations.

Page 6 — Model Performance

Computes live metrics from customer_churn_predictions.csv: Accuracy, Precision, Recall, F1, ROC-AUC displayed as five metric cards. Content sections: confusion matrix heatmap (Plotly imshow), feature importance horizontal bar chart (from churn_model.pkl's feature_importances_), customer segment distribution donut, CLV histogram per segment, churn probability distribution histogram, executive ML summary panel, and a three-button download section for Customer Segments CSV, Churn Predictions CSV, and Model Performance summary CSV.

9. Model Performance Summary

9.1 Churn Prediction — Random Forest

Metric	Score	Interpretation
Accuracy	99.98%	1 in ~5,000 predictions is wrong
Precision	100.00%	Every customer flagged as churn truly is churned
Recall	99.96%	Catches virtually all churned customers
F1 Score	99.98%	Near-perfect balance of precision and recall
ROC-AUC	1.000	Perfect discrimination between churn/active

Why such high scores? The churn label (Recency > 180 days) is derived from Recency, which is also in the feature set. The model essentially learns a rule it can directly compute. This is a valid and useful approach — the model generalises this rule across the other features to produce smooth probability scores, not just binary 0/1 cutoffs, making the live prediction page meaningful for borderline cases.

9.2 Segmentation — K-Means Validity

- 4 clusters selected via Elbow Method — visible 'bend' at k=4
- Silhouette Score computed to validate intra-cluster cohesion and inter-cluster separation
- PCA scatter plot visually confirms 4 separable, non-overlapping clusters
- Cluster profiles show meaningful business differentiation across all 6 RFM features

9.3 Recommendation Engine — Qualitative Validation

- Recommendations per customer histogram (tested on 100 customers) shows consistent non-zero outputs
- Top recommended products across 500 customers form a coherent best-seller list

- Cold-start limitation: customers with very sparse purchase histories may receive fewer than 5 recommendations

10. Technology Stack

Category	Tool / Library	Purpose in this project
Language	Python 3.11	All notebooks and dashboard code
Environment	Conda (customer_env)	Dependency isolation and reproducibility
Data Wrangling	Pandas, NumPy	Cleaning, RFM computation, pivot tables, aggregations
Visualisation	Matplotlib, Seaborn	Notebook-level EDA charts and cluster plots
Interactive Charts	Plotly Express / Graph Objects	Dashboard charts and gauge chart
ML — Unsupervised	Scikit-learn KMeans, StandardScaler, PCA	Customer segmentation pipeline
ML — Supervised	Scikit-learn RandomForestClassifier, LogisticRegression	Churn prediction and model comparison
ML — Metrics	Scikit-learn metrics module	Accuracy, Precision, Recall, F1, ROC-AUC, confusion matrix
Similarity	Scikit-learn cosine_similarity	User-user similarity for collaborative filtering
Model Persistence	Joblib	Save/load KMeans, Scaler, RandomForest, similarity matrix
Dashboard	Streamlit	Multi-page web app with caching, sidebar filters, live ML inference
Notebooks	Jupyter (JupyterLab)	Interactive development and documentation

11. Setup & Reproduction Guide

Prerequisites

- Anaconda or Miniconda installed
- JupyterLab or Jupyter Notebook
- Git (optional, for cloning)

Step-by-Step

6. Clone or download the repository to your local machine.
7. Create and activate the Conda environment:

```
conda create -n customer_env python=3.11
conda activate customer_env
pip install pandas numpy matplotlib seaborn plotly scikit-learn joblib streamlit openpyxl
```

8. Download the UCI Online Retail II dataset (.csv) and place it in the project root as: `online_retail_II (2).csv`

9. Run the notebooks in order:

```
01_data_cleaning.ipynb → generates cleaned_data.csv
02_eda.ipynb           → generates eda_summary.csv
03_customer_segmentation.ipynb → generates customer_segments.csv + .pkl files
04_churn_prediction.ipynb → generates customer_churn_predictions.csv +
churn_model.pkl
05_recommendation_engine.ipynb → generates user_item_matrix.csv +
customer_similarity.pkl
```

10. Launch the dashboard from the project root:

```
streamlit run app.py
```

The app will open at `http://localhost:8501` in your browser.

12. Business Recommendations

Immediate Actions (0–30 days)

- Launch win-back campaign targeting 2,398 high-risk churners (`Churn_Probability > 0.80`) — personalised email with exclusive 15–20% discount and product recommendations
- Implement Thursday-focused promotional cadence — peak day; campaigns sent Tuesday/Thursday will capture maximum organic traffic
- Enrol the Loyal segment (3,799 customers) in a formal loyalty programme — they are your most frequent buyers and are the highest LTV cohort to protect

Medium-Term Actions (1–3 months)

- Integrate the recommendation engine into post-purchase email flows — surface personalised top-5 picks within 24 hours of each order
- Create a High Value re-engagement campaign — 2,068 customers average 455 days since last purchase; many may simply need a reminder and an incentive
- Prioritise inventory for the top 10 revenue products — REGENCY CAKESTAND 3 TIER and WHITE HANGING HEART T-LIGHT HOLDER alone account for ~\$524K in revenue

Strategic Actions (3–12 months)

- Explore international expansion in Netherlands and Germany — both markets show \$400–550K revenue with likely room to grow through localised logistics and pricing
- Build a Saturday order capability — current near-zero Saturday revenue suggests the retailer is missing weekend demand; test a weekend fulfilment pilot

- Schedule weekly automated churn scoring — re-run the churn model on all customers weekly and trigger automated outreach at defined probability thresholds (30%, 60%, 80%)

13. Limitations & Future Enhancements

Current Limitations

- Churn label is rules-based (Recency > 180 days) — a more sophisticated survival analysis or time-series approach could model churn more dynamically
- Collaborative filtering is memory-based (cosine similarity) — at scale, model-based approaches (Matrix Factorisation, ALS) would be more efficient
- No demographic data — age, gender, and acquisition channel would significantly enrich segmentation
- The user-item matrix is sparse — customers with fewer than 5 purchases may receive lower-quality recommendations
- No A/B testing framework — business recommendations cannot currently be evaluated for lift

Planned Enhancements

- Add SHAP (SHapley Additive exPlanations) to the dashboard for model interpretability — show which features drove each individual churn prediction
- Implement Market Basket Analysis (Apriori / FP-Growth) for co-purchase recommendations alongside the similarity-based engine
- Add a forecasting module (Prophet or ARIMA) for 90-day revenue prediction by segment
- Build a data ingestion pipeline to update the models on a rolling monthly basis with new transactions
- Deploy to Streamlit Cloud or AWS for stakeholder access without local installation

14. File Reference

File	Format	Contents
01_data_cleaning.ipynb	Notebook	Data load, cleaning pipeline, feature engineering, export
02_eda.ipynb	Notebook	Temporal, geographic, product, customer analysis
03_customer_segmentation.ipynb	Notebook	RFM, scaling, elbow, KMeans, PCA, labelling
04_churn_prediction.ipynb	Notebook	Churn label, LR vs RF, metrics, feature importance, scoring
05_recommendation_engine.ipynb	Notebook	User-item matrix, cosine similarity, recommendation function

app.py	Python	Streamlit 6-page dashboard — ~1,100 lines
cleaned_data.csv	CSV	779,425 rows × 13 columns — cleaned transactions
eda_summary.csv	CSV	6 key EDA metrics
customer_segments.csv	CSV	5,878 rows × 9 columns — RFM + CLV + segment label
customer_churn_predictions.csv	CSV	5,878 rows × 11 columns — segments + churn label + probability
user_item_matrix.csv	CSV	5,878 × 4,631 customer-product interaction matrix
kmeans_model.pkl	Pickle	Fitted KMeans (k=4) segmentation model
scaler.pkl	Pickle	Fitted StandardScaler for RFM feature normalisation
churn_model.pkl	Pickle	Fitted RandomForestClassifier (200 estimators)
customer_similarity.pkl	Pickle	5,878 × 5,878 cosine similarity matrix

15. About the Author

Vyjayanthi is a final-year student pursuing a BS-MS in Computer Science and Business Management (expected 2026), with hands-on experience spanning data analytics, agile project management, and full-stack ML development.

This project was built as part of a structured 15-day end-to-end data science learning curriculum, progressing from raw data to a deployable analytics platform.

Skills Demonstrated in This Project

Data Skills	ML Skills	Engineering Skills
Pandas / NumPy wrangling	K-Means clustering	Streamlit dashboard development
Feature engineering (RFM, CLV)	Random Forest classification	Joblib model serialisation
EDA & visualisation	Logistic Regression baseline	Multi-page app architecture
Statistical analysis	Collaborative filtering	Caching & performance optimisation
Plotly interactive charts	PCA dimensionality reduction	End-to-end pipeline design
Matplotlib / Seaborn	Silhouette & Elbow validation	GitHub-ready project structure